# Dense Retrieval with Low-Rank Adaptation for Scalable Multi-Tenant Search

Francesca Pezzuti[1,*], Giulio Capecchi[1,*], Cesare Campagnano[2,*], Antonio Mallia[3,*] and Nicola Tonellotto[1,*]

[1]University of Pisa, Italy
[2]Independent researcher
[3]Seltz, United States

**Abstract**

Search systems often rely on dense retrieval models to support semantic retrieval at scale. However, on unseen domains they often struggle and require domain-specific fine-tuning. In multi-tenant search, where a single system serves many tenants, i.e., user groups with domain-specific data and queries, fine-tuning is unsustainable: replicating models for many tenants exhausts GPU memory, increases compute costs, and causes frequent disk-to-GPU swaps. Resource-constrained organisations are thus forced to choose between performance and system scalability. To tackle this, we use Low-Rank Adaptation to equip a single model with multiple tenant-specific adapters. Our approach cuts per-tenant GPU memory footprint up to 99.5% while losing less than 4% mean nDCG@10 and less than 3% mean R@1000 on six BEIR datasets, enabling effective and scalable multi-tenant search.

**Keywords**

Low-Rank Adaptation, Dense Retrieval, Multi-tenant Search

## 1. Introduction

A core component of modern search systems is *dense retrieval* [1, 2, 3, 4, 5], where queries and documents are mapped into learned embeddings that support semantic matching beyond lexical overlap. As such, dense retrieval provides the essential foundation for performing semantic search at scale.

At the same time, an increasing number of organisations, particularly small and medium-sized enterprises, rely on *multi-tenant search platforms* [6, 7, 8]. In this setting, each tenant contributes its own proprietary corpus and query stream, and the platform must provide high-quality retrieval for all tenants while sharing a common infrastructure. The challenges are manifold: new tenants must be accommodated quickly, storage and memory consumption must remain low, queries must be processed with low latency, and the effectiveness must be maintained despite the heterogeneity of tenant domains [8].

A central question in this context is how to deploy dense retrievers in practice. Several strategies are possible. One option is to use a single general-purpose model in a *tenant-agnostic zero-shot* fashion. This approach is extremely cheap and scales effortlessly, but dense retrievers are known to generalise poorly, yielding low effectiveness on unseen domains [9, 10]. A second option is to train a single *multi-tenant fine-tuned* model across all tenants. While this avoids per-tenant storage overhead, it is computationally expensive, must be repeated whenever a new tenant arrives, and is susceptible to catastrophic forgetting [11]. A third option is *tenant-specific fine-tuning*, where each tenant is assigned its own dedicated model. Although this provides strong effectiveness, it is unsustainable at scale: storing and serving many full models dramatically increases memory and infrastructure costs, and fine-tuning entire model backbones for every new tenant is slow and resource-intensive.

*IIR 2026: 16th Italian Information Retrieval Workshop, November 12–14, 2026, Rome, Italy*
*Corresponding author.
francesca.pezzuti@phd.unipi.it (F. Pezzuti); g.capecchi2@studenti.unipi.it (G. Capecchi); cesare.campagnano@gmail.com (C. Campagnano); antonio@seltz.ai (A. Mallia); nicola.tonellotto@unipi.it (N. Tonellotto)
0009-0005-2364-2043 (F. Pezzuti); 0000-0002-8362-274X (C. Campagnano); 0000-0002-7817-6140 (A. Mallia); 0000-0002-7427-1001 (N. Tonellotto)
© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

Taken together, these limitations highlight the need for a deployment strategy that lies between zero-shot and tenant-specific fine-tuning: one that enables lightweight adaptation of dense retrieval models to individual tenants, while retaining the scalability of a shared backbone. This requirement naturally points to *parameter-efficient fine-tuning* (PEFT), a family of methods for efficiently adapting large models to new downstream tasks. In contrast to fine-tuning the entire models, PEFT methods include adapters [12], prefix- and prompt-tuning [13, 14], BitFit [15], and Low-Rank Adaptation (LoRA) [16], offering different trade-offs between efficiency and effectiveness. Despite their popularity, PEFT methods have been scarcely explored in information retrieval: most prior work on adapting dense retrievers relies on full fine-tuning or knowledge distillation [17, 18].

In this work, we study tenant-specific LoRA adapters as a deployment strategy for scalable multi-tenant dense retrieval. The key idea is to retain a shared retrieval backbone while training lightweight, independently loadable adapters for each tenant. Among PEFT methods, LoRA is well suited in this setting: it keeps tenant-specific parameters compact and independently loadable, freezes the backbone, injects low-rank updates into existing linear layers, and controls the efficiency-expressiveness trade-off through its rank parameter. Prior work further suggests that LoRA can retain competitive effectiveness despite this reduced parameter footprint [19, 20].

Our results on six BEIR datasets [21], using three dense retrievers, show that LoRA-based deployment achieves scalable, cost-efficient, and effective multi-tenant search, offering a practical middle ground between zero-shot and tenant-specific fine-tuning.

Our main contributions are:

- We analyse the deployment challenges of dense retrieval in multi-tenant search, highlighting the limitations of existing deployment strategies (zero-shot, multi-tenant fine-tuning, and tenant-specific fine-tuning).
- We propose a practical approach based on *tenant-specific LoRA adapters*, enabling lightweight adaptation of a shared dense retriever.
- We conduct experiments on six BEIR datasets and three retrievers, showing that tenant-specific LoRA adapters can achieve effectiveness comparable to full fine-tuning while substantially reducing the per-tenant model-parameter footprint and tenant-specific model-weight loading time onto the GPU.
- We release code to support reproducibility.

## 2. Background

In this section, we introduce dense retrieval, discuss alternative deployment strategies for multi-tenant search, and finally describe Low-Rank Adaptation, the parameter-efficient approach we use to adapt retrievers across tenants.

### 2.1. Dense Retrieval

In dense retrieval [1, 2, 3, 4, 5], queries and documents are represented as continuous embeddings produced by neural encoders fine-tuned for search, typically with *contrastive learning* [22, 23, 24]. Relevance is estimated by a vector similarity function (e.g., dot product or cosine similarity) between the query and document embeddings, and the top-$k$ most similar documents are retrieved.

In this work, we focus on encoder-only biencoder retrievers, which encode queries and documents independently with the same transformer model (symmetric architectures) [17, 25, 26]. Examples include Contriever [25], E5 [26], and TAS-B [17]. Alternative formulations exist: encoder–decoder models like T5-based retrievers [27], decoder-only models like RepLLaMA [28], and asymmetric dual encoders (e.g., DRAGON [29]) that employ different encoders for queries and documents.

We next discuss how dense retrievers can be deployed in multi-tenant search scenarios.

### 2.2. Deployment Strategies for Multi-tenant Search

In multi-tenant search, where a single search platform serves many distinct *tenants*, dense retrievers can be deployed in different ways. The most cost-effective and scalable solution is to serve all tenants with a single deployed model, fine-tuned on a general-purpose dataset. This *tenant-agnostic zero-shot* strategy (zs) requires no further adaptation when accommodating a new tenant and introduces no additional inference overhead. However, dense retrievers are known to generalise poorly across domains [9, 10]. Thus, zs often delivers unsatisfactory effectiveness in multi-tenant scenarios where tenants operate in narrow or specialised domains.

An alternative strategy is *multi-tenant fine-tuning* (MTFT), where a single dense retriever model is iteratively fine-tuned on the data of all tenants. While MTFT avoids the need to host separate models for each tenant, it requires continuous fine-tuning to incorporate new tenants while also preserving previously acquired knowledge [30]. In practice, this approach is prone to *catastrophic forgetting* [31, 32, 19], is computationally expensive and technically challenging — particularly because of discrepancies between domains [11] — making it unsustainable in multi-tenant search. For these reasons, we do not pursue MTFT further in this work.

The limitations of single-model approaches motivate the need for tenant-specific solutions. In the *tenant-specific fine-tuning* (TSFT) strategy, a separate dense retriever is fine-tuned for each tenant on its own domain data. Although TSFT can improve effectiveness by tailoring models to individual tenants, it substantially increases hardware requirements, as each model must be potentially loaded into GPU memory, ultimately limiting scalability. Furthermore, accommodating a new tenant requires fine-tuning an entire pre-trained model, which is time-consuming and resource-intensive.

Taken together, these limitations highlight the need for a lightweight way to adapt dense retrieval models across tenants, which we address through *Low-Rank Adaptation* [16].

### 2.3. Low-Rank Adaptation

LoRA is a parameter-efficient alternative to full fine-tuning: it keeps the pre-trained model fixed and learns small low-rank updates for selected linear layers.

Let $W \in \mathbb{R}^{d_o \times d_i}$ denote the weight matrix of a *linear layer* $y = Wx$ where $d_i$ and $d_o$ are the *fan-in* and *fan-out* of the layer, respectively, and $x \in \mathbb{R}^{d_i}$ and $y \in \mathbb{R}^{d_o}$ are its input and output vectors. During standard fine-tuning, all weights in $W$ are updated; in large neural models with many such layers, this process is computationally expensive.

LoRA improves the efficiency of weight updates by injecting a low-rank, trainable matrix $\Delta W \in \mathbb{R}^{d_o \times d_i}$ on top of the weight matrix $W$, i.e., $y = (W + \Delta W)x$. The matrix $\Delta W$ is factorised as the product of two matrices: $y = Wx + (BA)x$, where the pair $B \in \mathbb{R}^{d_o \times r}$ and $A \in \mathbb{R}^{r \times d_i}$ forms an *adapter*, and $r \ll \min\{d_i, d_o\}$ is called *rank*. During fine-tuning, the $d_i d_o$ weights in $W$ are frozen, while the $r(d_i + d_o) \ll d_i d_o$ parameters of $BA$ are trained from scratch.

This property makes LoRA well suited to multi-tenant search, where each tenant can be assigned a lightweight adapter fine-tuned on its own data, while sharing the same dense retrieval backbone.

## 3. Methodology

We now describe our approach for dense retrieval with LoRA adapters for multi-tenant search and the methodology we use to assess the scalability and the effectiveness of our proposed approach.

In our approach for multi-tenant search, we train tenant-specific, lightweight adapters on top of a shared biencoder model, using each tenant's dataset. Specifically, we uniformly insert LoRA adapters with rank $r$ into all the linear layers of each encoder layer in a pre-trained biencoder model. We refer to this configuration as LORA-$r$. We leave to future work the investigation of layer-specific ranks.

The main scalability advantage of our approach over tenant-specific fine-tuning is that it avoids duplicating the model for each tenant. Since LoRA often achieves effectiveness comparable to fine-tuning [19, 20], we expect our approach to perform on par with tenant-specific fine-tuning while

outperforming zero-shot approaches. Another important advantage is that accommodating a new tenant requires training only lightweight adapters rather than an entire pre-trained model, thereby reducing the number of tenant-specific trainable parameters.

To assess the efficiency of our proposed approach at inference time, we compute how many parameters need to be stored to accommodate multiple tenants, namely the *parameter count*, in different scenarios. The parameter count directly impacts disk storage and GPU usage, and can affect tenant-switching overhead at serving time. We focus on symmetric biencoders, whose backbone contains $L$ encoder layers. Each encoder includes several linear layers: let $D_i$, resp. $D_o$, denote the sum of the input, resp. output, dimensions of all layers in a single encoder. Then, the number of parameters of the additional adapters in a LoRA-$r$ configuration is $M_\ell = rL(D_i + D_o)$.

Let $P_{\mathrm{X}}(t)$ the number of model parameters, namely the *parameter count*, that must be stored when the multi-tenant search system uses strategy X to serve $t$ tenants, where $\mathrm{X} \in$ TSFT, LoRA-$r$.

The tenant-specific fine-tuning approach has a parameter count

$$P_{\text{TSFT}}(t) = tM,$$

since for each tenant we store an entire biencoder with $M$ parameters[1]. Instead, the parameter count of our proposed LoRA-$r$ approach is

$$P_{\text{LoRA-}r} = M + tM_\ell = M + trL(D_i + D_o),$$

since we store a single biencoder model and a LoRA-$r$ adapter per tenant.

While the proposed LoRA-$r$ approach has a clear benefit in terms of the total number of backbone and adapter parameters that must be stored compared with storing multiple models, each fine-tuned on a different dataset for a separate tenant, the LoRA-$r$ approach also affects per-tenant and average effectiveness of the overall multi-tenant search system. In the following, we assess the performance of our proposed approach in terms of both space efficiency and effectiveness.

## 4. Experiments

We conduct experiments to answer the following research questions:

**RQ1** To what extent does LoRA-$r$ reduce the model-parameter memory footprint w.r.t. TSFT?

**RQ2** To what extent does LoRA-$r$ reduce inference-time tenant-specific weight loading time compared to that of TSFT?

**RQ3** How does the retrieval effectiveness of LoRA-$r$ compare to that of TSFT and ZS?

We do not investigate the retrieval-index storage consumption or end-to-end query latency under dynamic tenant-switching workloads, as these depend on system-level choices and our goal is to isolate the model-side scalability trade-off between tenant-specific fine-tuning and LoRA-based adaptation.

### 4.1. Experimental Setup

We evaluate LoRA-$r$ with ranks $r \in \{4, 8, 16\}$, initialising the adapters using the PiSSA method [33], which accelerates convergence and improves effectiveness. Our baselines are TSFT and ZS; we exclude MTFT due to its major drawbacks. As dense retrievers, we use three widely used MS MARCO [34] fine-tuned symmetric biencoder models: Contriever[2] [25], E5-base-v2[3] [26], and TAS-B[4] [17]. We

[1]Assuming that we fine-tune the same biencoder model for all tenants.
[2]https://huggingface.co/facebook/contriever-msmarco
[3]https://huggingface.co/intfloat/e5-base-v2
[4]https://huggingface.co/sebastian-hofstaetter/distilbert-dot-tas_b-b256-msmarco

leave asymmetric settings to future work. We use six BEIR datasets as a benchmark proxy for multi-tenant search: each dataset is treated as a tenant, as it defines a distinct corpus, query distribution, and retrieval task. We select the BEIR datasets with publicly available training and test splits: Fever (fact-checking), SciFact (scientific fact-checking), NFCorpus (biomedical), FiQA (finance), HotpotQA (multi-hop QA), and NQ (QA). We fine-tune using the InfoNCE loss [24], with temperature 0.05, batch size 16, 4 gradient-accumulation steps, and learning rate $5 \cdot 10^{-5}$. We apply cosine decay with 500 warm-up steps and optimise with AdamW [35]. For each query, we sample 4 hard negatives[5] and 4 in-batch negatives. We stop training early when nDCG@10 on a sampled validation subset does not improve by at least 0.01 for 5 consecutive evaluations, or after 5,000 train steps. We evaluate average nDCG@10 and R@1000 effectiveness across tenants on the test splits of the BEIR datasets. We quantify per-tenant GPU memory footprint of LoRA-$r$'s adapters and TSFT's duplicated models. We also evaluate tenant-specific weight-loading overhead, i.e., the cost of switching between tenants at serving time. To ensure a fair comparison, all measurements are performed on the same Nvidia L40 GPU. For LoRA-$r$, we assume that LoRA modules are already injected[6].

The source code to reproduce our experiments is publicly available on GitHub[7].

## 4.2. Experimental Analysis

To answer our research questions, we now analyse the results of our experiments.

### 4.2.1. RQ1. Model-Parameter Memory Efficiency

We begin by investigating the extent to which LoRA-$r$ reduces the model-parameter memory footprint compared to tenant-specific fine-tuning (TSFT). We first analyse how the parameter count of LoRA-$r$ scales with the rank $r$ of the adapters used for the biencoders under study.

The Contriever and E5-base-v2 (E5) models are built on the BERT-base architecture, consisting of $L = 12$ encoder layers and 109M parameters [25, 26, 37], whereas TAS-B is built on DistilBERT, which has 66M parameters and $L = 6$ encoder layers [17, 38]. For both these architectures, each encoder layer contains an *attention layer* with key, query, value and output projection matrices with dimensions $d_i = d_o = 768$, and a *feed-forward* layer with two linear layers with dimensions $d_i = 768, d_o = 3072$ and $d_i = 3072, d_o = 768$, respectively [39]. Thus, the combined sum of the input and output dimensions of an encoder layer is $D_i + D_o = 13,824$. Given these values, we obtain that the number of parameters of a single LoRA-$r$ adapter is:

$$M_\ell^{\text{Contriever}} = M_\ell^{\text{E5}} \approx 166K \times r \qquad \text{and} \qquad M_\ell^{\text{TAS-B}} \approx 83K \times r.$$

Thus, when using our LoRA-$r$ approach with the Contriever and E5 biencoders, accommodating a new tenant requires storing an additional $0.15r\%$ of the original parameter count, whereas for TAS-B, the overhead is $0.13r\%$. Indeed, for $r \in \{4, 8, 16\}$, this per-tenant overhead falls in the range $[0.60\%, 2.42\%]$ for Contriever and E5, and in the range $[0.52\%, 2.08\%]$ for TAS-B. Therefore, although the additional *parameter count* of LoRA-$r$ grows linearly with the adapter rank $r$, for commonly used LoRA ranks, it remains very modest, i.e., always less than 3%.

For TSFT, instead, the per-tenant overhead is constant, since accommodating a new tenant requires duplicating the full biencoder. In contrast, LoRA-$r$ deploys tenant-specific LoRAs, reducing the per-tenant additional parameters by 97.6%-99.5% and making LoRA-$r$ substantially more scalable than TSFT in terms of model-memory resource efficiency.

As shown in Figure 1, this considerable reduction in parameter count $P_X(t)$ for accommodating $t$ tenants directly translates into scalability benefits of our LoRA-$r$ approach in terms of GPU memory footprint as the number of tenants grows.

[5] We mine hard negatives using the TopK-PercPos method [36] ($K = 2,000$, $\alpha = 0.05$).
[6] Adapter injection is a one-time setup cost of about 5–6s in our implementation.
[7] https://github.com/fpezzuti/lora-dense-retrieval

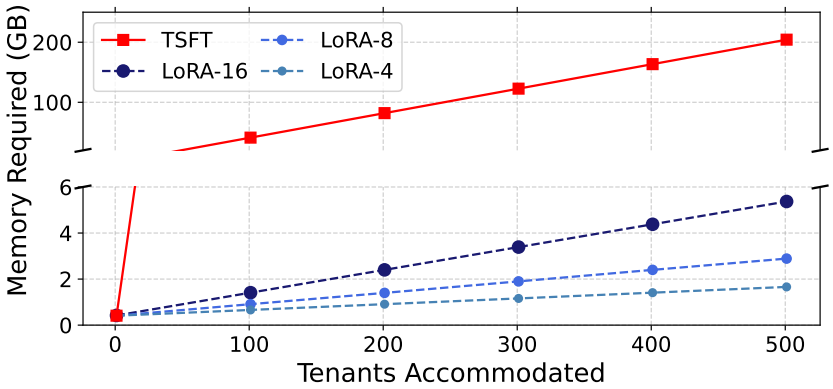



**Figure 1:** GPU memory footprint in gigabytes ($y$-axis) as the number of tenants grows ($x$-axis), of TSFT and LORA-$r$ for retrievers based on BERT-base (Contriever and E5).

**Table 1**
Average tenant-specific weight loading time in milliseconds over 1k repetitions. For TSFT, this corresponds to loading the full tenant-specific model; for LORA-$r$, it corresponds to loading the tenant-specific adapter, with the shared backbone already resident on the GPU. Percentages indicate relative reductions w.r.t. TSFT.

| **Tenant-specific Weight Loading Time (ms)** | | | | |
|---|---|---|---|---|
| Approach | Contriever | | E5 | |
| TSFT | 361 | | 299 | |
| LORA-*4* | 2.719 | (-99%) | 2.709 | (-99%) |
| LORA-*8* | 2.725 | (-99%) | 2.688 | (-99%) |
| LORA-*16* | 2.733 | (-99%) | 2.746 | (-99%) |

For instance, LORA-*16*, the most resource-demanding LORA-$r$ variant considered, requires roughly 5GB of model-parameter memory to support 500 tenants, whereas TSFT requires approximately 200GB to serve the same number of tenants. Moreover, the differences between LORA-$r$ variants are relatively small compared to the substantial scalability gains over TSFT. Similar results hold for TAS-B.

To answer RQ1, our results suggest that, from the perspective of model-parameter storage, LoRA enables multi-tenant search systems to support substantially more tenants on the same GPU than tenant-specific fine-tuning. Overall, our proposed LORA-$r$ method reduces the per-tenant parameter overhead by 97% w.r.t. TSFT.

#### 4.2.2. RQ2. Tenant-specific Weight Loading Time

We next evaluate the overhead of loading tenant-specific weights at serving time, which affects how efficiently the system can switch among tenants.

In Table 1, we show the time required to load tenant-specific weights for the TSFT baseline and our LORA-$r$ approach. As expected, loading the lightweight LORA-$r$ adapters is substantially faster than loading a full tenant-specific model: across both Contriever and E5, LORA-$r$ reduces tenant-specific loading overhead by approximately two orders of magnitude compared to TSFT. Furthermore, this latency remains stable across ranks, suggesting that the adapter rank $r$ has negligible impact on tenant-switching time.

To answer RQ2, our proposed LORA-$r$ deployment approach substantially reduces tenant-specific model-weight loading overhead compared with TSFT, suggesting that LoRA-based adaptation can simplify the model-side component of tenant switching.

**Table 2**
Average retrieval effectiveness of LoRA-*r*. Marker † denotes significant statistical non-equivalence w.r.t. TSFT (paired TOST, $p \leq 0.01$). We show percentage differences w.r.t. TSFT inside brackets. Bold denotes maxima per metric and model. Highlighted cells indicate the maximum across LoRA-*r* variants.

| **nDCG@10** | | | | | | |
|---|---|---|---|---|---|---|
| Approach | Contriever | | TAS-B | | E5 | |
| ZS | 0.5380 | | 0.5015 | | 0.5993 | |
| TSFT | **0.6263** | | **0.5841** | | **0.6535** | |
| LoRA-*16* | 0.6201 | (-1.0%) | 0.5714 | (-2.2%) | 0.6454 | (-1.2%) |
| LoRA-*8* | 0.6216 | (-0.7%) | 0.5658† | (-3.1%) | 0.6437 | (-1.5%) |
| LoRA-*4* | 0.6126 | (-2.2%) | 0.5612† | (-3.9%) | 0.6405 | (-2.0%) |
| **R@1000** | | | | | | |
| Approach | Contriever | | TAS-B | | E5 | |
| ZS | 0.8853 | | 0.8564 | | 0.9076 | |
| TSFT | **0.9302** | | **0.9114** | | **0.9406** | |
| LoRA-*16* | 0.9240 | (-0.7%) | 0.8949 | (-1.8%) | 0.9345† | (-0.7%) |
| LoRA-*8* | 0.9180 | (-1.3%) | 0.8939 | (-1.9%) | 0.9376 | (-0.3%) |
| LoRA-*4* | 0.9189 | (-1.2%) | 0.8873† | (-2.6%) | 0.9247 | (-1.7%) |

#### 4.2.3. RQ3. Effectiveness

We now address RQ3 by comparing the average effectiveness of our LoRA-*r* approach with that of TSFT, while also reporting ZS as a reference baseline.

In Table 2, for all three biencoder models considered, we report the average effectiveness measured across all tenants when equipped with rank-*r* tenant-specific LoRAs, i.e., LoRA-*r* with $r \in \{4, 8, 16\}$, and compare it with that of models specifically fine-tuned for individual tenants, that is, the TSFT approach. For reference, we also show the effectiveness of the ZS approach; note, however, that the percentage differences reported are relative to TSFT.

Across the evaluated models, TSFT consistently exhibits the highest nDCG@10 and R@1000. Yet, our LoRA-*r* variants consistently outperform ZS and remain competitive w.r.t. TSFT, with drops less than 4%. We then focus on how LoRA-*r* effectiveness varies with *r* and we identify the best-performing variant (denoted by a coloured background). As *r* decreases, and hence model expressiveness decreases, the drop in effectiveness w.r.t. LoRA-*16* is modest and sometimes LoRA-*r* with lower ranks outperforms LoRA-*16*. Thus, in line with findings from prior work [19, 20], we find that limiting adapter capacity can sometimes yield improvements.

To conclude on RQ3, for commonly used adapter ranks, LoRA-*r* consistently achieves competitive effectiveness relative to tenant-specific fine-tuning across the considered backbones and metrics.

## 5. Conclusions

To balance the competing demands of multi-tenant search, we proposed using Low-Rank Adaptation (LoRA) as a deployment-oriented strategy to train lightweight, tenant-specific adapters on top of a single shared pre-trained biencoder. Positioned between tenant-agnostic zero-shot and tenant-specific fine-tuning, our approach delivers effective retrieval while remaining cost-efficient and scalable from a model-parameter perspective, making it promising for scalable multi-tenant retrieval. Experiments on six datasets from the BEIR benchmark show that our method achieves effectiveness comparable to tenant-specific fine-tuning while reducing per-tenant model-parameter memory by up to 99.5% and substantially lowering tenant-switching overhead. Future work could explore tailoring LoRA configurations to individual tenants, employing non-uniform adapter injections, or extending the method to other neural search architectures.

## Acknowledgments

This work was partially supported by the FoReLab and CrossLab projects (Departments of Excellence) and the NVIDIA Academic Grants Program 2026.

## Declaration on Generative AI

During the preparation of this work, the authors used generative AI tools to improve the writing style and check grammar and spelling. The authors subsequently reviewed and edited the content and take full responsibility for the publication.

## References

[1] V. Karpukhin, B. Oguz, S. Min, P. S. Lewis, L. Wu, S. Edunov, D. Chen, W.-t. Yih, Dense Passage Retrieval for Open-Domain Question Answering., in: Proc. EMNLP, 2020, pp. 6769–6781.

[2] L. Xiong, C. Xiong, Y. Li, K.-F. Tang, J. Liu, P. Bennett, J. Ahmed, A. Overwijk, Approximate nearest neighbor negative contrastive learning for dense text retrieval, 2020. ArXiv:2007.00808.

[3] K. Lee, M.-W. Chang, K. Toutanova, Latent Retrieval for Weakly Supervised Open Domain Question Answering, in: Proc. ACL, 2019, pp. 6086–6096.

[4] C. Macdonald, N. Tonellotto, I. Ounis, On single and multiple representations in dense passage retrieval, in: Proc. IIR, 2021.

[5] A. Yates, R. Nogueira, J. Lin, Pretrained Transformers for Text Ranking: BERT and Beyond, in: Proc. WSDM, 2021, pp. 1154–1156.

[6] P. Dašić, J. Dašić, B. Crvenković, Applications of the search as a service (SaaS), in: Proc. PRASIC, volume 9, 2017, pp. 91–98.

[7] P. Dašić, J. Dašić, B. Crvenković, Service models for cloud computing: Search as a service (SaaS), IJET 8 (2016) 2366–2373.

[8] B. I. Ismail, R. Kandan, H. Ahmad, E. M. Goortani, M. Nizam Mohd Mydin, M. F. Khalid, O. H. Hoe, A Proposed Framework for Search as a Service on Private Cloud, in: Proc. ICOS, 2018, pp. 7–12.

[9] T. C. Rajapakse, M. de Rijke, Improving the Generalizability of the Dense Passage Retriever Using Generated Datasets, in: Proc. ECIR, 2023, pp. 94–109.

[10] Y. Zhu, H. Yuan, S. Wang, J. Liu, W. Liu, C. Deng, H. Chen, Z. Liu, Z. Dou, J.-R. Wen, Large Language Models for Information Retrieval: A Survey, ACM TOIS (2025).

[11] A. Royer, T. Blankevoort, B. Ehteshami Bejnordi, Scalarization for multi-task and multi-domain learning at scale, in: Proc. NeurIPS, volume 36, 2023, pp. 16917–16941.

[12] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, S. Gelly, Parameter-efficient transfer learning for NLP, in: Proc. ICML, 2019.

[13] X. L. Li, P. Liang, Prefix-Tuning: Optimizing Continuous Prompts for Generation, in: Proc. ACL-IJCNLP, 2021, pp. 4582–4597.

[14] B. Lester, R. Al-Rfou, N. Constant, The Power of Scale for Parameter-Efficient Prompt Tuning, 2021.

[15] Z. Ben Elad, Y. Goldberg, S. Ravfogel, BitFit: Simple Parameter-efficient Fine-tuning for Transformer-based Masked Language-models, in: Proc. ACL, 2022, pp. 1–9.

[16] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, LoRA: Low-Rank Adaptation of Large Language Models, in: Proc. ICLR, 2021.

[17] S. Hofstätter, S.-C. Lin, J.-H. Yang, J. J. Lin, A. Hanbury, Efficiently Teaching an Effective Dense Retriever with Balanced Topic Aware Sampling, in: Proc. SIGIR, 2021, pp. 113–122.

[18] T. Gao, X. Yao, D. Chen, Simcse: Simple contrastive learning of sentence embeddings, 2021. ArXiv:2104.08821.

[19] D. Biderman, J. Portes, J. J. Gonzalez Ortiz, M. Paul, P. Greengard, C. Jennings, D. King, S. Havens,

V. Chiley, J. Frankle, C. Blakeney, J. P. Cunningham, LoRA Learns Less and Forgets Less, TMLR (2024).
[20] C. Xin, Y. Lu, H. Lin, S. Zhou, H. Zhu, W. Wang, Z. Liu, X. Han, L. Sun, Beyond full fine-tuning: Harnessing the power of LoRA for multi-task instruction tuning, in: Proc. LREC-COLING, 2024, pp. 2307–2317.
[21] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, I. Gurevych, BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models, in: Proc. NeurIPS, 2021.
[22] L. Gao, Z. Dai, J. Callan, Rethink Training of BERT Rerankers in Multi-Stage Retrieval Pipeline, in: Proc. ECIR, 2021, pp. 280–286.
[23] M. Gutmann, A. Hyvärinen, Noise-contrastive estimation: A new estimation principle for unnormalized statistical models, in: Proc. AISTATS, 2010, pp. 297–304.
[24] A. van den Oord, Y. Li, O. Vinyals, Representation Learning with Contrastive Predictive Coding, 2019. ArXiv:1807.03748.
[25] G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bojanowski, A. Joulin, E. Grave, Unsupervised Dense Information Retrieval with Contrastive Learning, Proc. TMLR (2022).
[26] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, F. Wei, Multilingual e5 text embeddings: A technical report, 2024. ArXiv:2402.05672.
[27] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, P. J. Liu, Exploring the limits of transfer learning with a unified text-to-text transformer, JMLR 21 (2020) 1–67.
[28] X. Ma, L. Wang, N. Yang, F. Wei, J. Lin, Fine-Tuning LLaMA for Multi-Stage Text Retrieval, in: Proc. SIGIR, 2024, pp. 2421–2425.
[29] S.-C. Lin, A. Asai, M. Li, B. Oguz, J. Lin, Y. Mehdad, W.-t. Yih, X. Chen, How to Train Your DRAGON: Diverse Augmentation Towards Generalizable Dense Retrieval, 2023. ArXiv:2302.07452.
[30] G. I. Parisi, R. Kemker, J. L. Part, C. Kanan, S. Wermter, Continual lifelong learning with neural networks: A review, Neural Networks 113 (2019) 54–71.
[31] J. Lovón-Melgarejo, L. Soulier, K. Pinel-Sauvagnat, L. Tamine, Studying Catastrophic Forgetting in Neural Ranking Models, in: Proc. ECIR, 2021, pp. 375–390.
[32] L. Wang, X. Zhang, H. Su, J. Zhu, A Comprehensive Survey of Continual Learning: Theory, Method and Application, IEEE TPAMI 46 (2024) 5362–5383.
[33] F. Meng, Z. Wang, M. Zhang, Pissa: Principal singular values and singular vectors adaptation of large language models, Proc. NeurIPS 37 (2024) 121038–121072.
[34] P. Bajaj, D. Campos, N. Craswell, L. Deng, J. Gao, X. Liu, R. Majumder, A. McNamara, B. Mitra, T. Nguyen, M. Rosenberg, X. Song, A. Stoica, S. Tiwary, W. Tong, MS MARCO: A Human Generated MAchine Reading COmprehension Dataset, in: Proc. InCoCo@NIPS, 2016.
[35] I. Loshchilov, F. Hutter, Decoupled Weight Decay Regularization, in: Proc. ICLR, 2019.
[36] G. Moreira, R. Osmulski, M. Xu, R. Ak, B. Schifferer, E. Oldridge, NV-Retriever: Improving text embedding models with effective hard-negative mining, 2025. ArXiv:2407.15831.
[37] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: Pre-training of deep bidirectional transformers for language understanding, in: Proc. NAACL-HLT, 2019, pp. 4171–4186.
[38] V. Sanh, L. Debut, J. Chaumond, T. Wolf, DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter, in: Proc. EMC2@NIPS, 2019.
[39] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, in: Proc. NeurIPS, volume 30, 2017.